CONSIDERATIONS FOR TASK-AWARE AGENTS

Henry Xu
UC Berkeley
henryxu@berkeley.edu

Victor Chan
UC Berkeley
victorchan@berkeley.edu

December 16, 2019

ABSTRACT

With the rise of meta-learning as a means for generating more versatile and adaptive agents, there is strong potential for task-awareness to further empower agent decision-making. In this paper, we explore extensions to current task-aware agents such as PEARL and HSML to enhance learning efficacy. Specifically, we look at two main ideas: the application of curriculum learning to such task-aware algorithms in continuously parameterized environments with the goal of fostering novel links between tasks to improve learned skill transfer, and the application of model-based RL to PEARL to further improve sampling efficiency by making the algorithm fully off-policy. We find that the online, incremental nature of curriculum learning poses a barrier to the training of task encoders, leading to suboptimal performance; however, supplying the task parameters deterministically does yield noticeable and promising results. Model-based RL is useful for PEARL, but it only achieves high returns on simple environments.

1 Introduction

We explore two extensions to prior work on hierarchical meta-learning, specifically approaches that use meta-information about the task to inform agents' behavior. Algorithms of particular interest to us were Probabilistic Embeddings for Actor Critic RL (PEARL) [1] and Hierarchically Structured Meta Learning (HSML) [2], and how we could improve performance by reevaluating the assumptions they rely on.

First, we evaluate task sampling methodology by examining the effectiveness of curriculum learning in training these agents, exploring whether or not incrementally increasing task difficulty encourages formation of better policies if the agent is actively attempting to model (i.e., PEARL) or explicitly aware (i.e., HSML) of the task representation. Our main contribution in this area is the following:

- Adaptation of HSML, previously limited to supervised learning tasks, to continuously parameterized reinforcement learning tasks.
- Integration and assessment of task embedding strategies for PEARL, HSML, and SAC.
- Evaluation of teacher and task sampling strategies for PEARL and HSML.

We find a number of results of interest. First, there are significant benefits during training when HSML is provided a teacher that can adaptively sample tasks; however, these results do not carry as prominently over to evaluation as we would've hoped. On the other hand, augmenting a teacher-enhanced SAC with the task parameterization affords a noticeable boost during evaluation. Finally, PEARL surprisingly struggles with a teacher, likely due to its reliance on an encoder that performs suboptimally when not exposed to enough variety. Additional discussion of findings is provided in the Experiments section of the paper.

Our second extension combines model-based reinforcement learning with PEARL. One main advantage of PEARL with regards to efficiency is that its meta-training can be fully off-policy, relying on rollouts collected into a replay buffer. However, it still requires relatively on-policy rollouts to train its latent space encoders, which serves as a bottleneck for the algorithm.

We combined PEARL with Dyna to learn transition and reward models in the state space. These models, trained on data collected from the environment, were then used to generate the samples for both learning the latent space encoders

and meta-training. We find that on simple environments, Dyna-PEARL is able to perform on the same level as regular PEARL; moreover, once the model has finished training, data collection is much quicker. However, on more difficult environments with larger state and action spaces, Dyna-PEARL is unable to make significant progress. We ran a number of experiments with Dyna-PEARL, which are discussed further in this paper.

2 Related Work

2.1 Curriculum Learning

Curriculum learning was initially proposed as a means to optimize neural network training in supervised machine learning tasks by breaking a task down into easier subproblems that the learner could use as building blocks for the larger task at hand [3, 4]. The idea was adapted to reinforcement learning as interest in transfer learning grew, leading to the rise of curriculum generation strategies such as the one detailed in [5, 6].

Teacher-Student Curriculum Learning [7] experimented with a variety of teacher algorithms in discrete environments, which was expanded upon by [8] to tackle continuous environments in which large areas of the parameter space could be infeasible. We propose an extension to [8] through the exploration of task aware students who don't necessarily need to act as black boxes to the teacher.

Our definition of task-aware students apply to students who are able to use meta-information about the task at hand, e.g., through task embeddings, to enhance learning. Task2Vec proposes a framework for such an idea in the supervised domain [9]. In RL, the use of a task embedding network was explored in [10] for discrete tasks encoded as one-hot inputs. Our work most directly builds off PEARL [1] and HSML [2]. The former explored task representation through latent spaces supported by the encoding of trajectories as context, to which we add online task sampling for teacher support and the ability for the student to give feedback. The latter explored task embedding through clustering for supervised learning tasks–we adapt the methodology to a RL learning environment and introduce the use of a teacher to improve training.

2.2 Dyna

Standard reinforcement learning methods rely on the environment to provide the states and rewards that reward from an initial state and an action. Model-based reinforcement learning instead learns a model of the environment, including but not limited to transition and reward functions. For example, Dyna is a reinforcement learning framework that simultaneously learns the structure of an environment and generates a policy that achieves high rewards in that environment [11, 12]. Dyna learns both a transition model and a reward model, which it uses to generate the rollouts to train the policy with.

Since then, model-based RL has been extended in multiple directions. Dyna, a general framework for model-based RL, has been generalized to a wider variety of reinforcement learning algorithms, model types, and architectural choices. For example, it has been extended to deep Q-learning in Model-Based Acceleration. [13]. Other types of model-based RL algorithms include Multiple Model-based RL [14] and Guided Policy Search [15], which learn multiple models or policies of different parts of a task that are then combined to form a model and policy for the full task. Haarnoja et. al combines model-based RL to hierarchical RL, which, like PEARL, uses a latent space [16]. However, none of these methods extend model-based RL to meta-learning.

Clavera et al. does incorporate model-based RL into meta-learning with MB-MPO [17], but it does not use a latent space representation for multi-task learning. On the other hand, PEARL uses an encoder to learn a latent task representation [1], but it relies on on-policy samples to train the encoders. While PEARL is mainly off-policy, it is held back by the on-policy learning required to train the latent task representation.

3 The Continuous Teacher-Student (CTS) Framework with Task-Awareness

We start with some background to the continuous teacher-student framework outlined in [**?**] augmented by our task-awareness assumption. There are three main parts:

1. The student is able to interact with an episodic POMDP, each step observing observation o, taking action a and receiving reward r. Upon conclusion of the episode, the total reward is computed as the sum of rewards over all steps. Furthermore, the student has access to the task parameterization as specified by the teacher, and is able to use it to tune their actions accordingly. This can be in the form of a learned embedding, or a direct use of the parameterization.

Algorithm 1: Teacher-Enhanced PEARL

Require : Continuous task parameterization space, ALP-GMM Teacher initialized to work in the space, learning rates for SAC training

while *not done* **do**

  Request next batch of tasks from Teacher

  Perform PEARL update using only these tasks

  Update Teacher with agent rewards and skills

end

2. The teacher is able to assign tasks to the student specified by the tasks' parameterization, and receives feedback in the form of the student's reward. With a task-aware student, it is also able to receive feedback from the student about what student chooses to do with the task, e.g., information about the task embedding space. Our preliminary experiments incorporating this additional feedback yielded suboptimal results, but this may be an avenue of further exploration.

   To determine what the teacher should assign, we use the ALP-GMM algorithm described in [**?**]. The general idea of ALP-GMM is that we fit a Gaussian mixture model to the task parameterizations concatenated to the Absolute Learning Progress, the latter of which is defined for a particular task parameterization p as

$$alp_p = |r_p - r_{old}| \tag{1}$$

   where r_{old} is the reward of achieved by the student for the task specified by the closest task parameterization to p in euclidean space, and sample task parameterizations accordingly.

3. The task space is parameterized by a piecewise non-linear smooth function. The teacher has no prior knowledge of this function, and there may be areas of infeasibility or extraneous information.

4 Dyna-PEARL

In order to remove the reliance on on-policy learning from PEARL, we learn a dynamics model. Specifically, we use a neural network architecture to learn a transition function $\hat{p}(s, a)$ and a reward model $\hat{r}(s, a)$. During the meta-training phase, we gather rollouts from the environment and use these samples to train the dynamics model, which generates a next state s' and a reward r given a state s and action a. More concretely, we learn a function $f : (s, a) \rightarrow (s', r)$ from the rollouts. Once we have taken the requisite number of training steps for that iteration, we use the model, along with actions selected by the policy, to generate samples for the replay buffer, which is in turn used to train the policy.

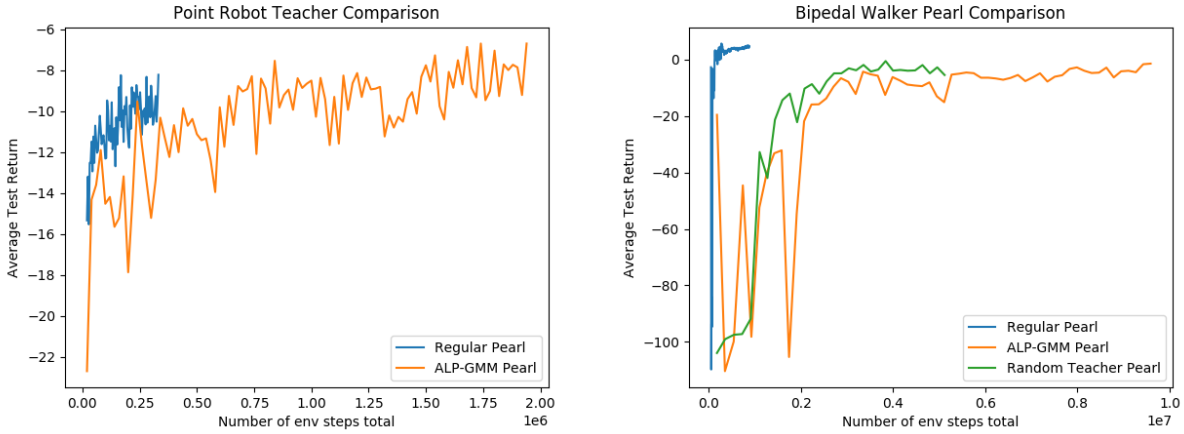
Our model, $f : (s, a) \rightarrow (s', r)$, is a standard feed-forward fully-connected neural network. It acts akin to an environment, taking in a state and action pair and producing a state and reward pair. It does not directly model terminal values, although these could be expressed through 0 reward. The network loss is taken as a weighted sum of the state loss and the reward loss.

Dyna-PEARL has many hyperparameters, including the size and number of layers of the model neural network, how many iterations to train the model for, the number of model training steps taken per iteration, whether to train the model in tandem with learning or before meta-learning, and the weight of the reward loss in the model network less.

5 Curriculum Learning Experiments

Our first set of experiments deal with the task sampling strategies of PEARL. As is, PEARL requires a predefined batch of sample tasks from which it can repeatedly sample and build out each of their respective train and encoding sample buffers. However, for a teacher to be able to assign new tasks as the student gains competency, we adopt an online approach to task sampling. Each iteration, instead of sampling from the existing batch of tasks, we query the teacher for a new batch and collect new train and encoding samples accordingly. Training of both SAC and the encoder is done exclusively on this new batch; however, we did also test various old task sampling schemes to minimize catastrophic forgetting as detailed in variants section. A general outline of the algorithm is in Algorithm 1.

Our second set of experiments implements Dyna-PEARL and tests different sets of hyperparameters on the algorithm. For the model network, we empirically chose hyperparameters of 3 layers and 64 nodes per layer. For the other hyperparameters, we ran different combinations to test the effects of each hyperparameter on the performance of PEARL.



(a) Point Robot Task Sampling Comparison

(b) Bipedal Walker Task Sampling Comparison

Figure 1: **Teacher Enhanced Pearl Results.** For the most part, the online approach to task sampling gives suboptimal results. A common theme seen also in later results is the added training burden of the encoder, which we believe to be one of the main culprits.

5.0.1 Results

We started with Point Robot to gauge general efficacy of our approach. While performance lagged behind regular PEARL, we believed the simplistic nature of the task gave regular PEARL and its exposure to a wide diversity of tasks upfront the advantage. However, even on a much more difficult task like Bipedal Walker, we saw noticeably degraded performance compared to regular PEARL, leading to investigation of strategies to increase the breadth of tasks trained on by the student. In fact, even a random teacher was more effective than ALP-GMM, which seemed to indicate to us diversity outweighed benefits from incremental skill development. See Figure 3 for performance details.

5.0.2 Variants

To foster diversity without requiring the sampling of completely new tasks, we tried occasionally reintroducing previously seen tasks into the batch of tasks the teacher assigned. This also allowed us to investigate if we were facing the problem of catastrophic forgetting. While performance did rise to match that of the random teacher, the returns still fell short of that of regular PEARL. See Figure 2a for performance details. We started to suspect the trouble wasn't so much with the teacher, but the nature of PEARL and the additional burden of the task latent space encoder which didn't benefit from only seeing a limited variety of tasks each training cycle. To address these concerns, we tried deterministically augmenting teacher-driven SAC next.

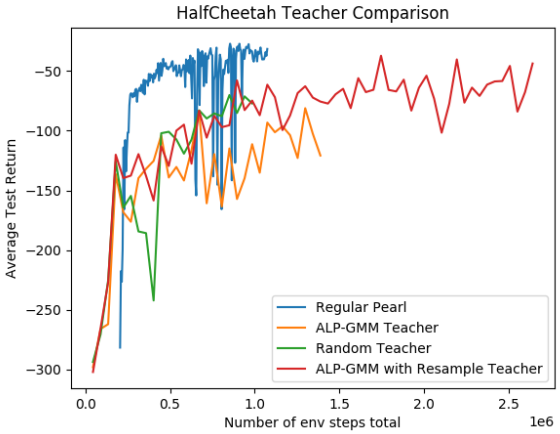
5.1 Task-Augmented Teacher-Driven SAC

To encode knowledge of the task at hand into teacher-driven SAC without an embedding model, we pursued an observation augmentation strategy in which the new observation o' within task with parameterization p and original observation o was defined as:

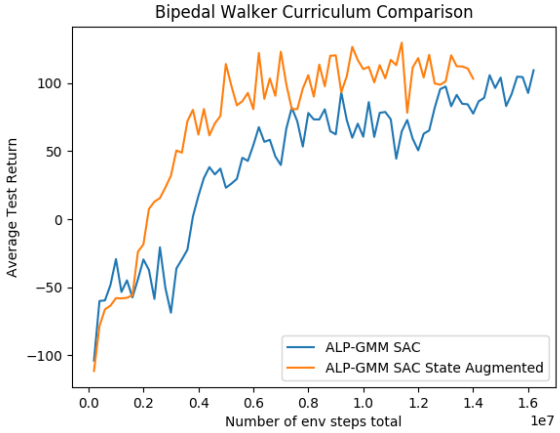
$$o' = [o|p] \tag{2}$$

5.1.1 Results

While the augmentation was straightforward, we saw significant performance improvements over non-augmented SAC as shown in 2b. The additional information afforded by the task parameterization allows the student to make more customized decisions pertaining to the environment at hand. When the augmented agent is visualized, we see greater within task consistency, and different behaviors as the tasks change, as opposed to the non-augmented agent who tries to tackle all the problems with the same initial approach. We did not test augmented non-teacher SAC due to computational constraints, but note that [**?**] does show ALP-GMM performs considerably better than a random task sampler. To further confirm our suspicions of the embedding model being the reason behind performance woes, we tried one last task-aware meta-learning variant, HSML.

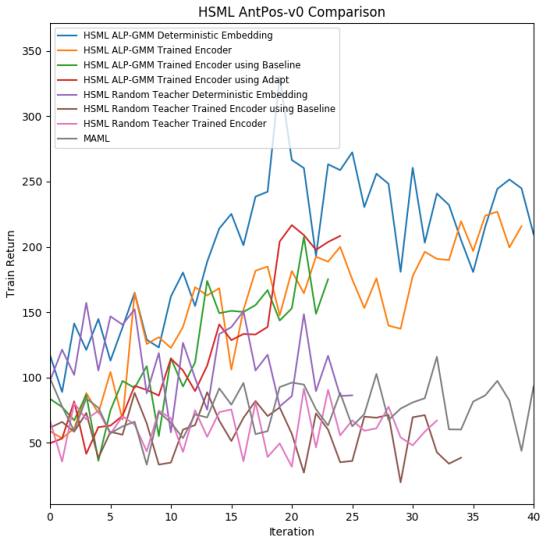


(a) Half Cheetah Variant Task Sampling Comparison

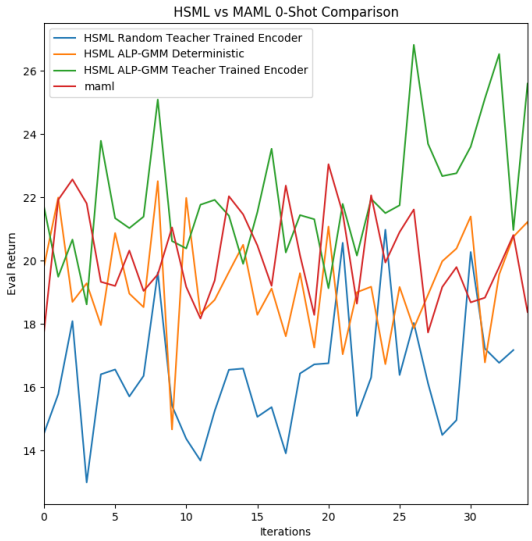


(b) Bipedal Walker Curriculum Learning Task-Augmented SAC Comparison

Figure 2: **Teacher Pearl Variant and Task Augmented Teacher SAC Results.** Believing limited task exposure each iteration was a problem, we tried a strategy where the teacher would resample previously seen tasks. Performance did noticeably increase, but not to where we would've liked. Swapping out the encoder for a simple deterministic approach in augmenting teacher-driven SAC yielded far more interesting results further explored in subsequent figures.



(a) Curriculum Learning HSML Train Return Comparison



(b) Curriculum Learning HSML Eval Return Comparison

Figure 3: **Curriculum Learning HSML Results.** We find that ALP-GMM produces train returns that are not commensurate with what it achieves during evaluation. However, HSML ALP-GMM with a Trained Encoder does seem to pull ahead during evaluation, which may be worth future investigation.

5.2 Curriculum Learning HSML

HSML was designed for supervision learning in mind, specifically image classification, so the first challenge was to adapt it to RL environments. To generate an task encoding from supervised learning data without meta-knowledge of the task, HSML feeds the labelled data through an RNN, not unlike how PEARL formulates task context using the trajectories it collects. To avoid exactly mimicking PEARL in this regard, we decided to directly use the task parameterization in our RL implementation of HSML. However, of particular interest in HSML is the passing of the encoding through a Tree-LSTM to identify clusters of related tasks, a functionality we preserved. Finally, we concatenate the hidden state of the Tree-LSTM to each of the observations from the task an train TPRO on this augmented observation and task cluster embedding space.

5.2.1 Results

Train
In terms of training performance, we see highest training performance in the deterministic version of HSML, that is with a random initialization and no training of the Tree-LSTM. We tried a number of training methods for the embedding model; however, their train performance lingered slightly below that of the deterministic approach. Note that compared to ALP-GMM HSML, MAML and the Random teachers struggle to make any headway at all. These results fall in line with our expectations, as the adaptive localization of the train tasks each iteration by ALP-GMM ensures that the student will tend to see higher returns because training on a specific subset of the tasks encourages the student to get better at those specific tasks faster. Alternatively, we can interpret the results as the student is more ready for the tasks it faces; however, to the extent this approach encourages better 0-shot evaluation is less clear.

Evaluation
While train performance for ALP-GMM HSML outpaced MAML significantly, we saw less of a difference during evaluation for the reasons stated above, since the teacher's selection of tasks during train-time did not match test-time task sampling distributions. As a result, 0-shot evaluation yielded similar results for deterministic HSML and MAML, with trained encoder ALP-GMM HSML actually a touch ahead, which surprised us given our prior belief. Due to limited computational resources, we are unable to determine if performance truly diverges later on, and view this as potentially an avenue of further exploration.

5.3 Dyna-PEARL

5.3.1 Point-Robot

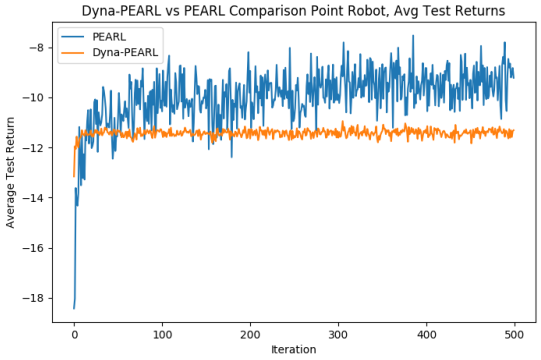
We first ran Dyna-PEARL on the Point Robot in order to test its performance against PEARL's performance. We trained the model for only the first 200 iterations, since empirically, the model loss did not improve over time if we trained the model any longer than that. Each iteration of PEARL took about 186 seconds on a Tesla V100 GPU, but after iteration 200 with Dyna-PEARL, the iteration time dropped to about 70 seconds. Almost all of the difference was in the sampling time. The results on Point Robot for Dyna-PEARL are close to the returns from PEARL, although not quite there.

For the Half-Cheetah environment, although the model loss decreases, the train and test returns do not increase. In fact, they do not even come close to the returns that PEARL achieves. We believe this is because of several factors. The hyperparameters are very difficult to tune, and checking the performance requires much waiting. In addition, while the model loss decreases, the model is not perfect, and PEARL, as a multi-task algorithm, might exploit imperfections in the model even more because it tries to generalize the tasks and improve at each one. Finally, the model does not incorporate terminal states, which could affect rollouts.

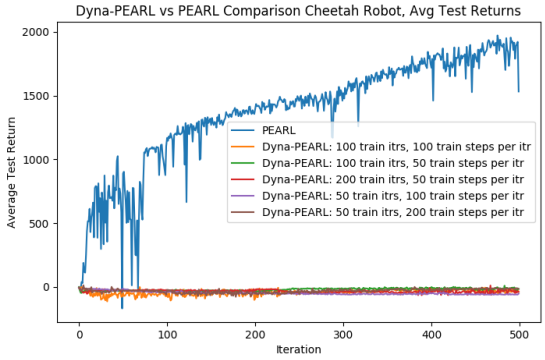
6 Conclusion

Task-aware agents such as PEARL and MAML have forged a very promising direction for metalearning. In this paper, we examine two extensions, curriculum learning and Dyna-PEARL, and evaluate their effects on training and sample efficacy.

We discover that while curriculum learning may not be as useful to agents who learn their own task representations, augmenting the observations of an agent, such as SAC, who otherwise would not internalize anything about the overarching task with the task parameterization does yield interesting results that we are interested in exploring as future work. Additionally, further examination of HSML in RL environments may be worth exploring for the potential insight into connections between tasks. Dyna-PEARL is effective for simple environments, but it does not have the



(a) Dyna-PEARL vs PEARL for Point Robot



(b) Dyna-PEARL vs PEARL for Half Cheetah

Figure 4: **Dyna-PEARL Results** We find that on a simple environment, the test performance of Dyna-PEARL is on par with the asynptotic performance of PEARL. However, on a more complicated environment such as Half Cheetah, Dyna-PEARL is unable to learn the complex model and thus does not achieve high performance.

power to model more complex environments, and as a result does not achieve similar performance to PEARL. In the future, different model-based reinforcement learning algorithms could be used with PEARL in order to increase the ability of model-based RL to accurately learn the structure of the environment.

References

[1] Kate Rakelly, Aurick Zhou, Deirdre Quillen, Chelsea Finn, and Sergey Levine. Efficient off-policy meta-reinforcement learning via probabilistic context variables. *CoRR*, abs/1903.08254, 2019.

[2] Huaxiu Yao, Ying Wei, Junzhou Huang, and Zhenhui Li. Hierarchically structured meta-learning. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

[3] Jeffrey L. Elman. Learning and development in neural networks: the importance of starting small. *Cognition*, 48(1):71 – 99, 1993.

[4] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, pages 41–48, New York, NY, USA, 2009. ACM.

[5] Carlos Florensa, David Held, Markus Wulfmeier, and Pieter Abbeel. Reverse curriculum generation for reinforcement learning. *CoRR*, abs/1707.05300, 2017.

[6] David Held, Xinyang Geng, Carlos Florensa, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents. *CoRR*, abs/1705.06366, 2017.

[7] Tambet Matiisen, Avital Oliver, Taco Cohen, and John Schulman. Teacher-student curriculum learning. *CoRR*, abs/1707.00183, 2017.

[8] Rémy Portelas, Cédric Colas, Katja Hofmann, and Pierre-Yves Oudeyer. Teacher algorithms for curriculum learning of deep rl in continuously parameterized environments, 2019.

[9] Alessandro Achille, Michael Lam, Rahul Tewari, Avinash Ravichandran, Subhransu Maji, Charless C. Fowlkes, Stefano Soatto, and Pietro Perona. Task2vec: Task embedding for meta-learning. *CoRR*, abs/1902.03545, 2019.

[10] Karol Hausman, Jost Tobias Springenberg, Ziyu Wang, Nicolas Heess, and Martin Riedmiller. Learning an embedding space for transferable robot skills. In *International Conference on Learning Representations*, 2018.

[11] Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *SIGART Bull.*, 2(4):160–163, July 1991.

[12] Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *In Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224. Morgan Kaufmann, 1990.

[13] Shixiang Gu, Timothy Lillicrap, Ilya Sutskever, and Sergey Levine. Continuous deep q-learning with model-based acceleration. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International*

Conference on Machine Learning, volume 48 of *Proceedings of Machine Learning Research*, pages 2829–2838, New York, New York, USA, 20–22 Jun 2016. PMLR.

[14] Kenji Doya, Kazuyuki Samejima, Ken-ichi Katagiri, and Mitsuo Kawato. Multiple model-based reinforcement learning. *Neural Computation*, 14(6):1347–1369, 2002.

[15] Sergey Levine and Vladlen Koltun. Guided policy search. In *International Conference on Machine Learning*, pages 1–9, 2013.

[16] Tuomas Haarnoja, Kristian Hartikainen, Pieter Abbeel, and Sergey Levine. Latent space policies for hierarchical reinforcement learning. *CoRR*, abs/1804.02808, 2018.

[17] Ignasi Clavera, Jonas Rothfuss, John Schulman, Yasuhiro Fujita, Tamim Asfour, and Pieter Abbeel. Model-based reinforcement learning via meta-policy optimization. *CoRR*, abs/1809.05214, 2018.